Smart Campus Analytics using Python Hadoop Streaming

MapReduce Laboratory: Mini Project Report

Aryan Awasthi

October 2026

1. Problem Understanding and Assumptions

The university wants operational insights from four activity logs: Wi-Fi sessions, canteen purchases, library visits and campus bus speeds. **SmartCampusAnalytics** answers six questions with six MapReduce jobs (ten MapReduce stages in total), written in Python and run with Hadoop Streaming. Each mapper reads CSV lines on `stdin` and prints `key\tvalue` lines. Each reducer reads those lines after they are sorted by key and prints one result per key.

Dataset. The worksheet gives each file's schema and one example row, but no data. I wrote `scripts/generate_data.py` to create a reproducible (seeded) dataset for one campus week, 10–14 Aug 2026. It contains 700 Wi-Fi sessions, 300 canteen transactions (594 item lines), 160 library visits (with matching EXIT rows) and 1,050 bus speed readings. A few deliberately malformed lines are added to every file to test how the jobs reject bad records.

Assumptions

#	Assumption
A1	`canteen.csv` has **one row per item**. A transaction with three items has three rows that share a `TransactionID`.
A2	In `library.csv`, the visit duration is recorded on the `ENTRY` row. `EXIT` rows have duration 0 and are left out, because counting them would halve every average.
A3	Job 1's "hour" is a **calendar hour** (`YYYY-MM-DD-HH`), as the hint says. Job 6's "hour" is the **hour of day** (`HH`), pooled over the week, so the result is a recurring congestion pattern such as "R04 at 08:00".
A4	The "overall average" in Job 5 is total minutes ÷ total visits over all visits. It is **not** the average of the per-student averages.
A5	A co-purchased pair is counted **once per transaction**, even if an item appears on two rows of that transaction.
A6	A record is malformed if it has too few fields, a non-numeric number or an unreadable timestamp. Such records are skipped and counted with a Hadoop counter. Blank lines are skipped silently.

Correction to the reference reducer. The Job 1 reducer in the worksheet treats each input line as a complete key group, with values joined by `;`. Hadoop Streaming does not group values

that way. It sends one line per value and puts lines with equal keys next to each other. With the reference code, the reducer would print one line per session instead of one per hour, and because the `print` sits outside the loop it would actually print only the last line. Every reducer in this project therefore uses the standard streaming pattern: keep a running total while the key stays the same, and print it when the key changes.

2. Project Structure

```
SmartCampus_MapReduce_Python/
├── data/            wifi_logs.csv  canteen.csv  library.csv
transport.csv
├── src/             job1..job6 mappers and reducers (19 files) +
job1_wifi_combiner.py
├── scripts/         generate_data.py  verify.py  top_n.py
compare_combiner.sh
├── output/          results of every job (produced by run_local.sh)
├── run_local.sh     runs all jobs locally:  map | sort | [combine] |
reduce
└── run_hadoop.sh    the same jobs on a cluster with hadoop-
streaming.jar
```

To run: `./run_local.sh` then `python3 scripts/verify.py`. For a cluster: `./run_hadoop.sh`.

3. Job Design

Job	Stage	Mapper output (key → value)	Reducer output
1 Peak Wi-Fi	1	`YYYY-MM-DD-HH` → `DataMB,DurationMin,1`	`TotalDataMB`, `TotalDurationMin`, `Sessions` per hour
2 Access point	1	`AccessPoint` → `DataMB,DurationMin,1`	totals, `Sessions`, `AvgSessionMin` = duration ÷ sessions
3 Canteen items	1	`Item` → `Quantity,Amount`	`TotalQuantity`, `TotalRevenue`
4 Co-purchase	1	`TransactionID` → `Item`	every sorted pair A|B → `1`
	2	A|B → `1` (identity)	`Transactions` = number of transactions containing the pair
5 Library anomaly	1	`StudentID` → `Duration,1` (ENTRY rows)	`StudentID` → `TotalMin,Visits`
	2 (global)	`ALL` → `TotalMin,Visits`	`OverallAvgMin`
	3	`StudentID` →	report line with `Flag=ANOMALY`

Job	Stage	Mapper output (key → value)	Reducer output
	(filter)	`Avg,Overall,Ratio`, emitted only if Avg > 1.5 × Overall	
6 Transport	1	`RouteID-HH` → `Speed,1`	`AvgSpeedKmph, Readings, Status` (CONGESTED if < 20)

Combiner. `job1_wifi_combiner.py` adds up the comma-separated numbers for each key and outputs them in the mapper's own format. The reducer therefore accepts its output unchanged. Every value it adds is a sum or a count, and those are associative and commutative. Averages are computed only in the reducer, from (sum, count). For that reason the same combiner is safe for Jobs 1, 2, 3, 4-stage-2, 5-stage-1, 5-global and 6.

Job 5 side input. The filter stage needs one number, the overall average, in every mapper. The global job's output is very small, so it is shipped to each mapper through the distributed cache (`-files global_avg.txt`) and the filtering happens map-side. The filter reducer only formats the result.

Job 6 partitioner. `run_hadoop.sh` uses `KeyFieldBasedPartitioner` with field separator `-` and `-k1,1`. Records are therefore partitioned by **RouteID only**, and all hours of one route go to the same reducer (see Q13).

4. Results

All results below come from `./run_local.sh`. `scripts/verify.py` recomputes every job directly from the CSVs without MapReduce. **All six jobs match (PASS).**

Job 1 — Peak Wi-Fi Usage (64 hourly keys)

Rank	Hour	TotalDataMB	TotalDurationMin	Sessions
1 (peak)	**2026-08-12 11:00**	**7,872.10**	**1,744**	**27**
2	2026-08-10 11:00	6,469.60	1,784	26
3	2026-08-11 11:00	6,130.50	1,327	22

Interpretation: the 11:00 hour is the busiest hour on several days, and the single heaviest hour is Wednesday 12 Aug at 11:00. This is the slot to protect when planning bandwidth upgrades, and to avoid for scheduling large software updates.

Job 2 — Access Point Utilisation

AP	TotalDataMB	TotalDurationMin	Sessions	AvgSessionMin
AP01	32,703.20	10,909	175	62.34
AP02	15,300.80	4,996	78	64.05
AP03	**34,825.60**	**11,291**	**181**	62.38

AP	TotalDataMB	TotalDurationMin	Sessions	AvgSessionMin
AP04	12,753.00	4,100	64	64.06
AP05	14,845.80	4,966	85	58.42
AP06	8,939.40	2,547	39	65.31
AP07	6,735.10	2,028	34	59.65
AP08	8,359.90	2,631	44	59.80

Interpretation: AP03 and AP01 together carry about 50% of all traffic. The average session length is similar everywhere (58–65 min), so the difference in load comes from the **number of users**, not from longer sessions. Adding a second access point near AP03 (and AP01) is the change that would help most.

Job 3 — Canteen Item Frequency

Item	Qty	Revenue (₹)	Item	Qty	Revenue (₹)
Burger	92	**11,040**	Muffin	83	3,735
Pasta	68	9,520	Juice	**120**	6,000
Sandwich	97	7,760	Coffee	112	4,480
Noodles	58	6,380	Tea	74	1,480
Water	67	1,340	Samosa	64	960

Interpretation: Juice and Coffee sell the most units, but Burger and Pasta bring in the most revenue. Ranking items by quantity alone would understate the importance of the high-value items.

Job 4 — Top Co-Purchased Pairs (45 distinct pairs from 392 pair records)

Rank	Pair	Transactions
1	Burger + Juice	42
2	Coffee + Sandwich	36
3	Juice + Pasta	33
4	Samosa + Tea	31
5	Coffee + Muffin	28

Interpretation: the top pairs are clear meal combinations. “Burger + Juice” and “Coffee + Sandwich” are natural candidates for combo offers, and placing these items next to each other would make them quicker to buy at the counter.

Job 5 — Library Usage Anomaly

Overall average visit = 14,736 min ÷ 160 visits = **92.10 min**, so the threshold is 1.5 × 92.10 = **138.15 min**.

Student	Avg visit (min)	Ratio to overall
S107	181.83	1.97×
S123	205.11	2.23×
S131	218.00	2.37×
S148	223.43	2.43×

Interpretation: 4 of 58 students stay in the library about twice as long as average. They are the heaviest users of reading-room seats, and they may be useful contacts for a survey on extended library hours or bookable study spaces.

Job 6 — Transport Congestion (70 route-hours, 14 congested)

Route	Congested hours (avg km/h)
R01	08 (17.84), 17 (19.03)
R02	08 (15.29), 09 (18.26), 17 (16.96), 18 (19.16)
R03	08 (18.83)
R04	**08 (13.08)**, 09 (15.37), 17 (14.57), 18 (16.17)
R05	08 (16.86), 09 (19.72), 17 (18.10)

Interpretation: every route is congested at 08:00, which is the morning rush. R04 is the slowest route overall and is congested in both rush hours. Adding R04 buses at 08:00–10:00 and 17:00–19:00, or staggering class start times, would ease the problem.

Malformed Records Rejected

File	Rejected (counter `SmartCampus:RejectedRecords`)
wifi_logs.csv	2 (non-numeric DataMB, missing fields)
canteen.csv	2 (Quantity = “two”, truncated row)
library.csv	2 (empty duration, line without commas)
transport.csv	1 (Speed = “fast”)

5. Expected Output Worksheet

Job	Expected Key	Expected Value	Student Result
Job 1 – Peak Wi-Fi	`YYYY-MM-DD-HH`	TotalDataMB, TotalDurationMin, Sessions	Peak = **2026-08-12-11**: 7,872.10 MB, 1,744 min, 27 sessions
Job 2 – Access Point	`AccessPoint`	TotalDataMB, TotalDurationMin,	Busiest = **AP03**: 34,825.60 MB, 11,291 min, 181 sessions, 62.38 min

Job	Expected Key	Expected Value	Student Result
		Sessions, AvgSessionMin	avg
Job 3 – Canteen	`Item`	TotalQuantity, TotalRevenue	Top revenue **Burger** ₹11,040 (92 units); top quantity **Juice** (120)
Job 4 – Co-Purchase	ItemA|ItemB (sorted)	Number of transactions	Top pair **Burger + Juice** = 42 transactions
Job 5 – Library	`StudentID`	AvgVisitMin, Ratio, ANOMALY flag	Overall avg 92.10 min; **S107, S123, S131, S148** flagged
Job 6 – Transport	`RouteID-HH`	AvgSpeedKmph, Status	14 / 70 route-hours congested; worst **R04-08** = 13.08 km/h

6. Student Implementation Worksheet

Q1. For Job 1, what should the Mapper output key be? Why? The key is the timestamp cut to the hour, `YYYY-MM-DD-HH` (e.g. `2026-08-10-09`). The framework groups records by key, so all sessions of the same hour reach the same reducer call, and the reducer can add them up. The fixed-width format also sorts in time order.

Q2. What is the Mapper output value? Can you design a more efficient representation? The value is `DataMB,DurationMin,1`: the two measures plus a session count of 1. More efficient options: (a) Use a **combiner** (or in-mapper combining with a dictionary), so that each mapper emits one partial `sum_data,sum_dur,count` per hour instead of one record per session. Here this cut the shuffle from 700 records to 67. (b) Drop the constant `1` and count lines in the reducer. This only works without a combiner, because a combined record stands for many sessions. (c) Use a compact binary or structured encoding (Writable/Avro, or integer KB instead of float text) to make each record smaller.

Q3. Why is the hour extracted from the timestamp instead of using the complete timestamp as the key? The question is about totals per hour. A full timestamp (to the minute) would make almost every record a separate key. The reducer would then just repeat the input, and finding the peak hour would need another grouping step. Truncating to the hour sets the grouping level to the one the analysis needs, and the shuffle has fewer keys.

Q4. Which operations can be performed using a Combiner? Operations that are **associative and commutative** and whose output has the same type as the mapper output: sum, count, min, max, and set union / distinct. In this project that means summing data, duration and sessions (Jobs 1 and 2), quantity and revenue (Job 3), pair counts (Job 4 stage 2), per-student minutes and visits (Job 5), and speed sums and counts (Job 6). An **average is not** combinable directly, because an average of averages is wrong. The fix is to combine (sum, count) and divide only in the reducer. Filtering against a threshold that depends on a global value, and the Job 4 stage-1 pairing (it needs every item of the transaction), cannot be done in a combiner.

Q5. For Job 2, define the key and value emitted by the Mapper. Key = `AccessPoint` (e.g. `AP03`). Value = `DataMB,DurationMin,1`. The reducer adds these to get total data, total duration and number of sessions, and computes average session duration = total duration ÷ sessions.

Q6. For Job 4, explain how you will generate every unique pair of products within a transaction. Stage 1 mapper emits `TransactionID → Item`, so all items of one basket meet in one reducer call. The reducer puts them in a **set** (removing duplicates), **sorts** them, and generates every 2-combination with `itertools.combinations(sorted_items, 2)`, i.e. a double loop `i < j`. For n items this gives n(n−1)/2 pairs, each emitted as `A|B → 1`. Stage 2 adds up the 1s for each pair.

Q7. Why must the product pair be sorted, e.g., (Coffee, Sandwich) rather than sometimes (Sandwich, Coffee)? MapReduce groups by the exact key string. `Coffee|Sandwich` and `Sandwich|Coffee` would be two different keys, may go to different reducers, and would each hold part of the count. Putting the pair in one fixed (alphabetical) order makes it a single key, so the count is complete.

Q8. For Job 5, how will you compute the overall average using more than one MapReduce job? 1. **Stage 1** (StudentID → duration,1): reducer outputs `StudentID → total_min, visits`. 2. **Global stage**: the mapper re-keys every student record to the single key `ALL`. The reducer adds the totals and visits and outputs `OverallAvg = Σtotal ÷ Σvisits`. This is the exact average over all visits, not an average of averages. A combiner keeps the single reducer's input small. 3. **Filter stage**: the overall average file is sent to every mapper (distributed cache `-files`, or `-cmdenv`). Each mapper computes the student average = total ÷ visits and emits only students above 1.5 × overall.

Several stages are needed because the threshold depends on **all** the data, and one pass cannot know the overall average while it is still reading records.

Q9. For Job 6, explain why RouteID and Hour form a composite key. Congestion depends on both **where** (route) and **when** (hour). Keying by route alone would average rush hour with off-peak and hide the congestion. Keying by hour alone would mix fast and slow routes. The composite key `RouteID-Hour` puts in one group exactly the readings whose average decides one congestion verdict. It also lets the partitioner use only its first part, the route (Q13).

Q10. Which jobs require more than one MapReduce stage? Justify your answer. - **Job 4 (2 stages):** the data must be grouped twice by different keys, first by `TransactionID` to build the pairs, then by pair to count them. A job has only one shuffle. - **Job 5 (3 stages):** per-student aggregation, then a global aggregation (the overall average needs every student), then a filter that uses that global value. - Jobs 1, 2, 3 and 6 need only one stage, because a single grouping key gives the answer. Picking the peak or top-3 hour from Job 1's output is a cheap post-processing step (or a small second job with one reducer).

Q11. What happens if a reducer receives millions of values for one key? That reducer becomes the **straggler**. The whole job waits for it while other reducers sit idle. Its input must be fetched over the network and merge-sorted on one node, which can cause disk spills, long garbage-collection pauses or out-of-memory errors if values are buffered in memory (e.g. a list

or set). Fixes: use a combiner, stream through values without storing them, or **salt** the key (`AP03#0..AP03#9`) to spread the load and then merge the partial results in a second small job.

Q12. Explain how data skew could affect the Smart Campus application. Campus activity is naturally uneven. AP03 (library) and AP01 have far more sessions than AP07. Lunch hours dominate canteen data. Popular items (Juice, Coffee) and pairs show up again and again. Rush-hour route keys have more readings. With hash partitioning, the reducers that get these hot keys do most of the work, so the job takes as long as the slowest reducer. In Job 5 stage 2, every record goes to the single key `ALL` on purpose, which is skew by design. It is only acceptable because stage 1 has already reduced the data to one record per student and a combiner pre-adds them.

Q13. Design a custom partitioning strategy for Job 6. Partition on the **RouteID part of the composite key** so that every hour of one route goes to the same reducer: `partition = hash(RouteID) mod R`. Each output file then holds a whole route's daily profile, ready for per-route reports. In Hadoop Streaming this needs no Java: `-partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner -D mapreduce.map.output.key.field.separator=- -D mapreduce.partition.keypartitioner.options=-k1,1` (used in `run_hadoop.sh`). If routes are very unequal in size, a **range or lookup partitioner** that assigns known busy routes to their own reducers (or partitions by hour during rush periods) balances the load better. Routes are few, so R ≤ number of routes is sensible.

Q14. What changes are necessary if the input contains malformed records? Parse defensively in the mapper: use the `csv` module, skip blank and header lines, check the number of fields, wrap number and timestamp parsing in `try/except`, and **skip the record instead of crashing** the task. A single exception would otherwise fail the map task and, after retries, the whole job. Count rejects with a Hadoop counter (`reporter:counter:SmartCampus,RejectedRecords,1` on stderr) so data quality is visible, and optionally write bad lines to a side output for inspection. Reducers should also validate their input when they read the output of an earlier stage. All mappers here do this; the counts are in section 4.

Q15. Suggest one additional campus analytics problem that can be solved using MapReduce. Student engagement / at-risk detection. Join Wi-Fi, library and canteen activity per student per week. The mapper emits `StudentID-Week → source:measure`, and a reducer-side join builds a weekly activity vector. A second job compares each student's week with their own history and flags sudden drops in campus presence, so student-support services can reach out early. A simpler alternative is **classroom occupancy estimation**: count distinct students per access point per lecture slot, and compare with room capacity to find over-booked or empty rooms.

7. Viva Questions — Prepared Answers

Question	Answer
Three major phases of MapReduce?	**Map** (records → key-value pairs), **Shuffle & Sort** (partition, transfer and sort by key, group values), **Reduce** (aggregate each key group).

Question	Answer
Mapper output vs Reducer output?	Mapper output is intermediate, unsorted per record, written to local disk and shuffled. Reducer output is the final result per key, written to HDFS (`part-*` files).
Why sort Mapper output before the Reducer?	So all values of a key arrive together and the reducer can process one key group at a time with a single pass and little memory. It also gives output sorted by key.
What is a Combiner; when is it safe?	A "mini-reducer" run on each mapper's local output to reduce shuffle volume. Safe only for associative and commutative operations whose output matches the mapper's output format. It may run 0, 1 or many times, so results must not depend on it.
What is a Partitioner?	It decides which reducer gets each key (default `hash(key) mod R`). All records with the same key must go to the same reducer.
Can a job have zero reducers?	Yes (`mapreduce.job.reduces=0`). It is a map-only job, with no shuffle, and mapper output is written straight to HDFS. Useful for filtering and cleaning, e.g. Job 5's filter could be map-only.
Multiple reducers?	Keys are split among them by the partitioner. Each produces its own sorted `part-r-0000N` file. Keys are not globally sorted across files, so top-N needs a final merge. I checked that 3 reducers give the same Job 1 result as 1.
What is a composite key?	A key made of several fields (e.g. `RouteID-Hour`) to group at a finer level. It is also used in secondary sort, where only part of it is used for partitioning and grouping.
Handling duplicate records?	Emit a unique record ID (e.g. `TransactionID+Item` or a hash of the line) as the key, and let the reducer keep one value per key. Within a group, use a set (as in Job 4).
Why multiple jobs?	One job has one grouping. Tasks that need two different groupings (Job 4) or a global value before filtering (Job 5) need chained jobs.
What is data locality?	Running map tasks on the node that already holds the HDFS block, so the program moves to the data instead of the data moving over the network.
Effect of a very large key group?	One reducer gets overloaded: straggler, spills, possible out-of-memory, longer job time (see Q11).
Reducing network traffic in the shuffle?	Combiners or in-mapper combining, compact keys and values, map-output compression (`mapreduce.map.output.compress=true`), filtering and projecting early in the mapper.
Testing a Mapper independently?	Pipe sample lines into it, e.g. `cat data/wifi_logs.csv` piped to `python3 src/job1_wifi_mapper.py`. Check the key-value lines, including malformed inputs. Then add `sort` and the reducer to simulate the full job. `verify.py` checks the end results.
Scaling to millions of records?	Store input in HDFS (splittable, possibly compressed) and run on the cluster with more mappers and reducers. Use combiners, a balanced partitioner and key salting for skew. Stream values in reducers and

Question	Answer
	keep no large lists. Use map-output compression. The Python code needs no change.

8. Extension Challenges Completed

Challenge	Status / evidence
Combiner for the Wi-Fi job	`src/job1_wifi_combiner.py`, used for Jobs 1–3, 4-s2, 5, 6.
Top three hours instead of all	`scripts/top_n.py` → `output/job1_top3_hours.txt` (section 4).
Custom partitioner for transport	`KeyFieldBasedPartitioner` on RouteID in `run_hadoop.sh` (Q13).
Malformed records + rejected count	2 / 2 / 2 / 1 rejected, Hadoop counters, `output/rejected_records.txt`.
With vs without combiner	Over 4 simulated map tasks, Job 1 shuffles **700** → **67** records (−90%) with identical output. The combiner pre-adds each hour inside each map task, so only one record per (map task, hour) crosses the network.
Multiple reducers	Job 1 with 3 hash-partitioned reducers (23/19/22 keys) merges to output **identical** to 1 reducer.

9. Conclusion

The six jobs show the main MapReduce design patterns: aggregation by a derived key (Jobs 1 and 6), summarisation (Jobs 2 and 3), pair generation with a second grouping (Job 4), and a global statistic followed by filtering (Job 5). The key choice decides the result in every job. For example, truncating the timestamp makes the hourly totals possible, sorting the pair makes it one key, and the composite route-hour key shows the congestion. Combiners, partitioning and defensive parsing make the jobs efficient and robust. The analysis points to clear operational actions: add Wi-Fi capacity near AP03 and AP01 at 11:00, offer Burger+Juice and Coffee+Sandwich combos, follow up with four heavy library users, and add R04 buses during rush hours.